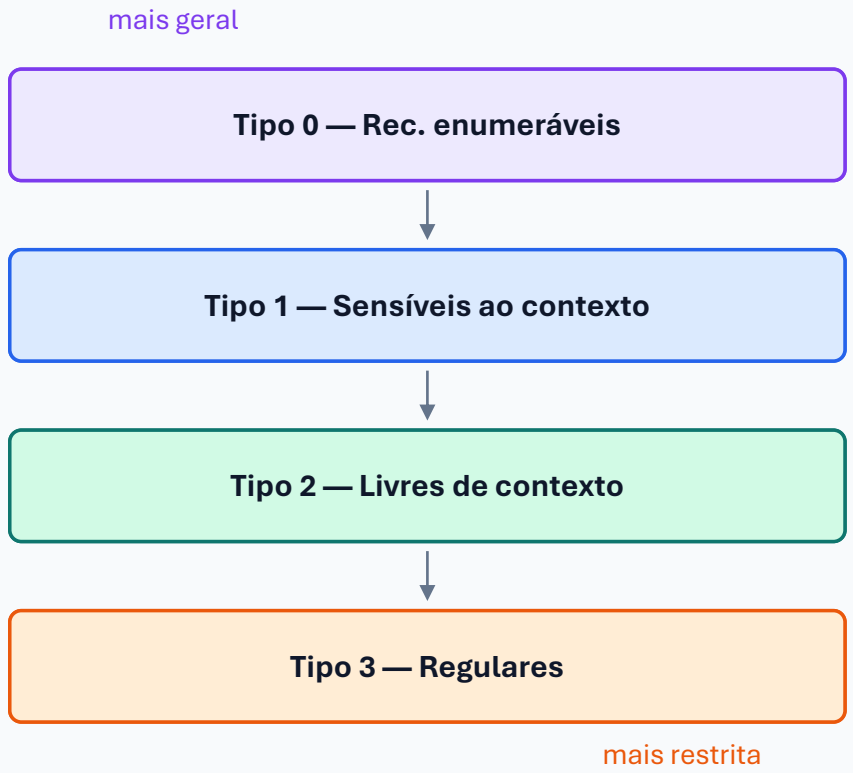
Gramáticas, decidibilidade e compiladores

Visão geral

Nesta aula, vamos ligar dois blocos importantes da disciplina: a teoria das linguagens formais e a construção de compiladores. A ideia é entender como gramáticas classificam linguagens, como problemas podem ser decidíveis ou não, e como esses conceitos aparecem nas fases reais de um compilador.



- Distinguir gramáticas sensíveis ao contexto, linguagens recursivas e noções de decidibilidade
- Identificar a arquitetura e as fases de um **compilador**

Recurso: JFLAP – ferramenta acadêmica gratuita

Objetivos de aprendizagem

O que dominar

Ao final da aula, o estudante deverá diferenciar gramáticas sensíveis ao contexto, linguagens recursivas e noções de decidibilidade. Também deverá reconhecer a arquitetura geral de um compilador, suas principais fases, a função de cada etapa e a relação entre análise formal e processamento de programas.



Ferramentas gratuitas ajudam a visualizar conceitos abstratos

Por que estudar isso em compiladores?

Motivação

Compiladores não são apenas programas que traduzem código. Eles verificam se um texto segue regras, constroem estruturas internas, detectam erros e geram outra representação executável. Por isso, gramáticas, linguagens e decidibilidade ajudam a explicar tanto o que pode ser automatizado quanto os limites dessa automação.



tokens → AST → tipos → otimização → assembly

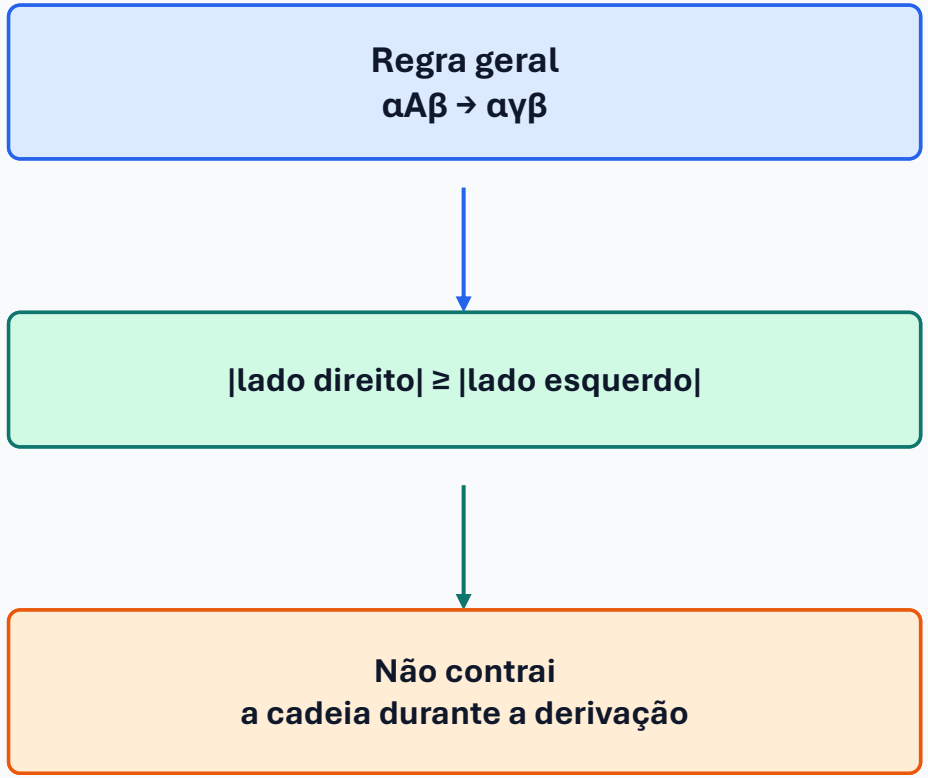
Diagnósticos e tabela de símbolos acompanham as fases

Recurso: LLVM Kaleidoscope – tutorial de front-end

Revisão: gramática e linguagem

Conceitos base

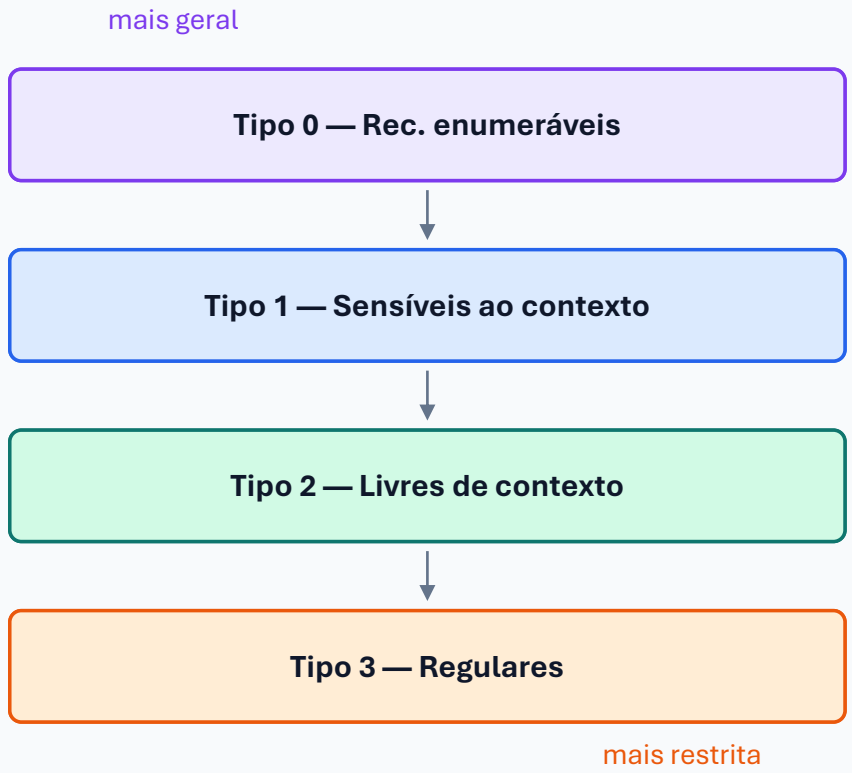
Uma gramática é um conjunto de regras usado para gerar cadeias de símbolos. Uma linguagem é o conjunto de todas as cadeias que obedecem a essas regras. Em computação, essas cadeias podem representar palavras, expressões, comandos, programas ou formatos de dados usados por sistemas.



Hierarquia de Chomsky

Classificação

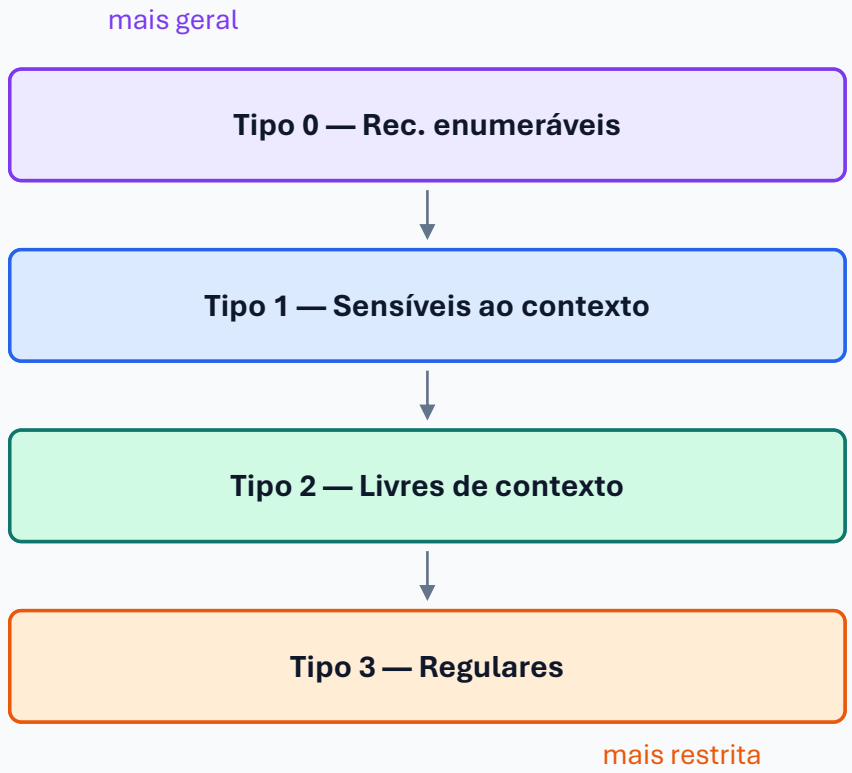
A hierarquia de Chomsky organiza linguagens pelo tipo de gramática e pelo poder do reconhecedor correspondente. Linguagens regulares são as mais restritas, depois vêm as livres de contexto, as sensíveis ao contexto e, por fim, classes ainda mais gerais ligadas a máquinas de Turing.



Onde entram as gramáticas sensíveis ao contexto?

Tipo 1

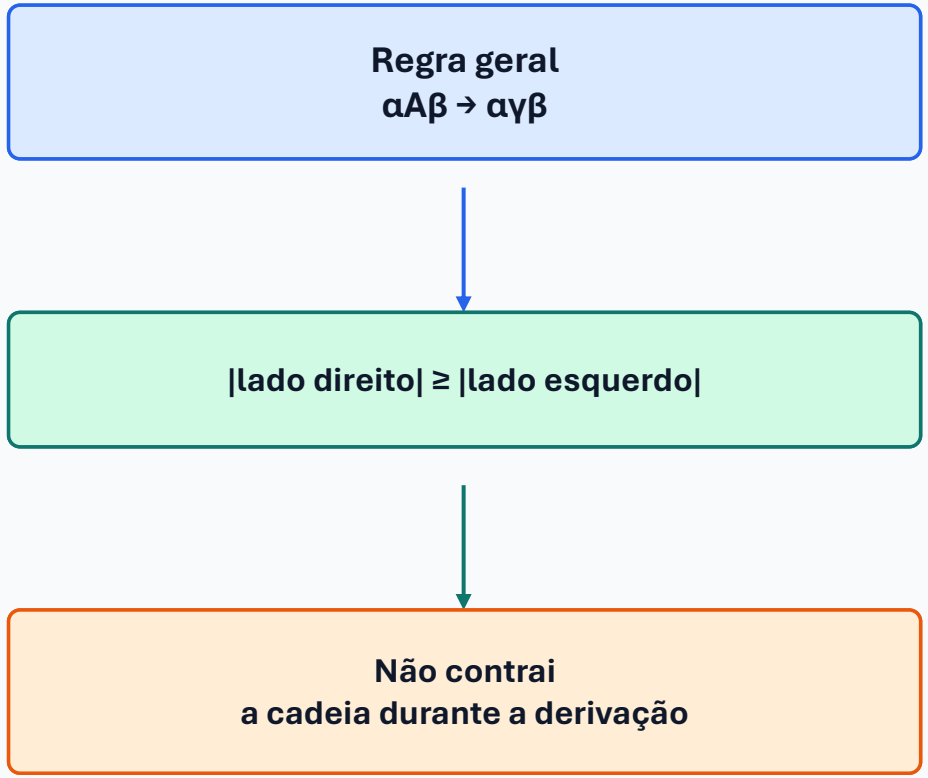
Gramáticas sensíveis ao contexto ocupam o tipo 1 da hierarquia. Elas são mais poderosas que gramáticas livres de contexto, pois conseguem impor dependências múltiplas entre partes da cadeia. Esse poder extra permite modelar padrões que exigem contagem ou relacionamento simultâneo entre grupos de símbolos.



Definição intuitiva de CSG

Regras dependem do contexto

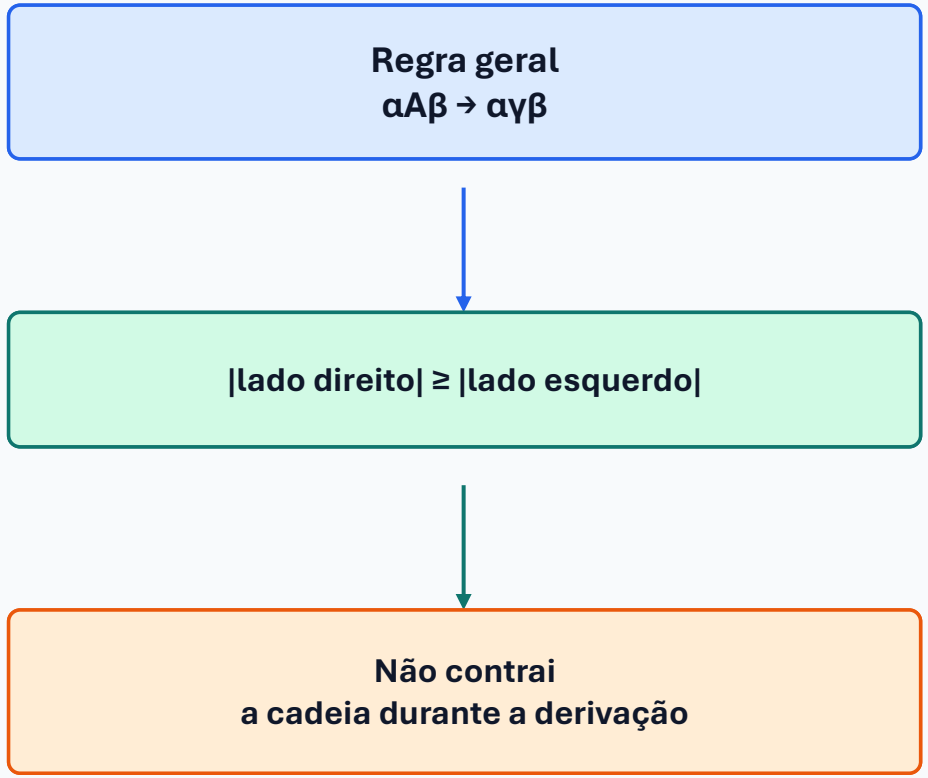
Em uma gramática sensível ao contexto, uma variável pode ser reescrita considerando os símbolos ao seu redor. A regra não trata apenas “troque A por algo”, mas pode significar “troque A por algo quando A estiver entre certos símbolos”. Isso permite controlar relações mais ricas.



Produções não contrativas

Restrição central

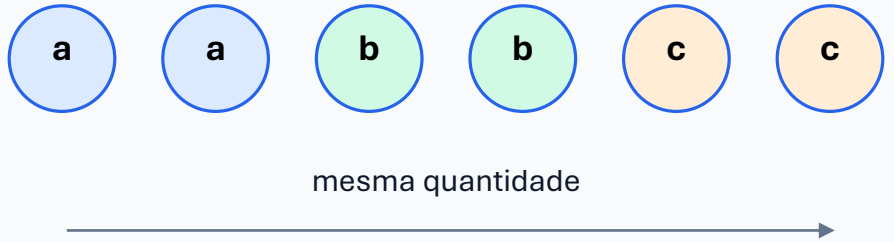
A restrição mais importante das gramáticas sensíveis ao contexto é que as produções não devem encurtar a cadeia. Em geral, o lado direito da regra precisa ter tamanho maior ou igual ao lado esquerdo. Assim, a derivação cresce ou mantém tamanho, evitando reduções arbitrárias.



Exemplo clássico: $a^n b^n c^n$

Dependência tripla

A linguagem L = { $a^n b^n c^n$ | n ≥ 1 } contém cadeias como abc, aabbcc e aaabbbccc. Ela exige que a quantidade de a, b e c seja exatamente igual. Essa dependência entre três blocos mostra por que precisamos de modelos mais fortes que GLC.

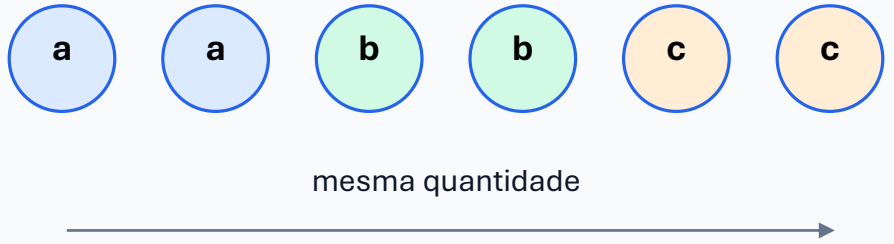


$$L = \{ a^n b^n c^n \mid n \geq 1 \}$$

Por que isso não é livre de contexto?

Intuição

Gramáticas livres de contexto conseguem controlar bem pares de dependências, como a quantidade de a em relação à quantidade de b. Porém, quando a cadeia exige três blocos sincronizados, a pilha única não é suficiente de forma geral. Por isso, $a^n b^n c^n$ é exemplo típico fora das GLC.

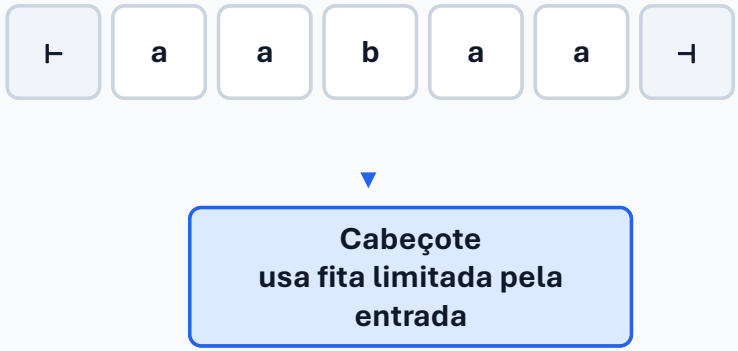


$$L = \{ a^n b^n c^n \mid n \geq 1 \}$$

CSG e autômatos linearmente limitados

Modelo reconhecedor

As linguagens sensíveis ao contexto são reconhecidas por autômatos linearmente limitados. Esse modelo pode ser entendido como uma máquina de Turing cuja fita de trabalho fica limitada por uma quantidade proporcional ao tamanho da entrada. Há mais memória que em uma pilha, mas não memória ilimitada.

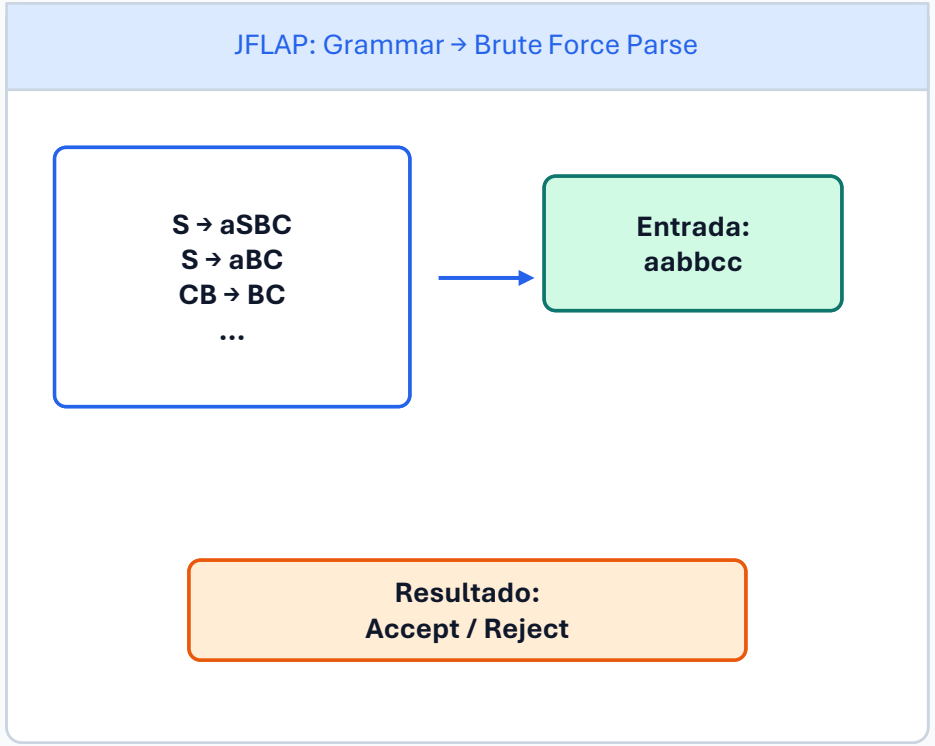


LBA = MT com memória linear

JFLAP como apoio visual

Ferramenta prática

O JFLAP permite criar gramáticas, autômatos e máquinas de Turing em uma interface visual. Para este conteúdo, ele ajuda a experimentar gramáticas mais expressivas e a observar derivação ou rejeição de cadeias. A ferramenta é útil porque torna conceitos abstratos mais concretos.

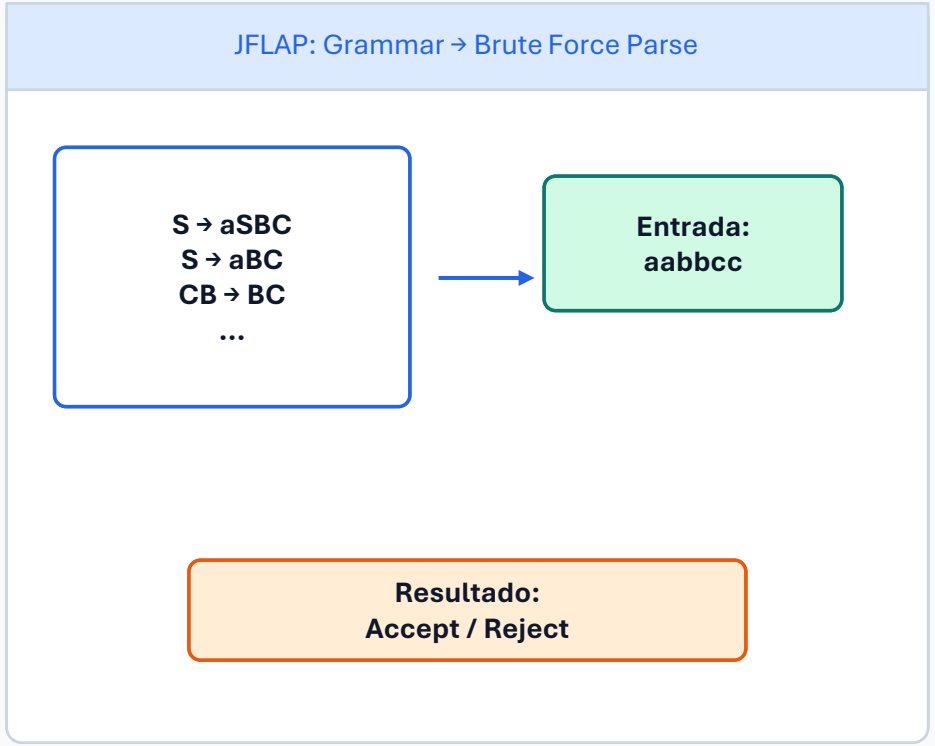


Recurso: JFLAP – ferramenta acadêmica gratuita

Passo a passo: criar uma CSG no JFLAP

Laboratório

Para praticar, abra o JFLAP, escolha Grammar e cadastre as produções de uma gramática. Use letras maiúsculas para variáveis e letras minúsculas para terminais. Depois, use a opção de parser para testar cadeias pequenas, como abc e aabbcc, observando se são aceitas.



Recurso: JFLAP – ferramenta acadêmica gratuita

Cuidado com busca exaustiva

Limite prático

Ao testar gramáticas complexas, o parser por força bruta pode explorar muitos caminhos. Cada escolha de produção pode criar novos ramos de busca. Por isso, uma gramática curta nem sempre é rápida de testar; às vezes uma construção mais guiada produz resultados mais previsíveis.



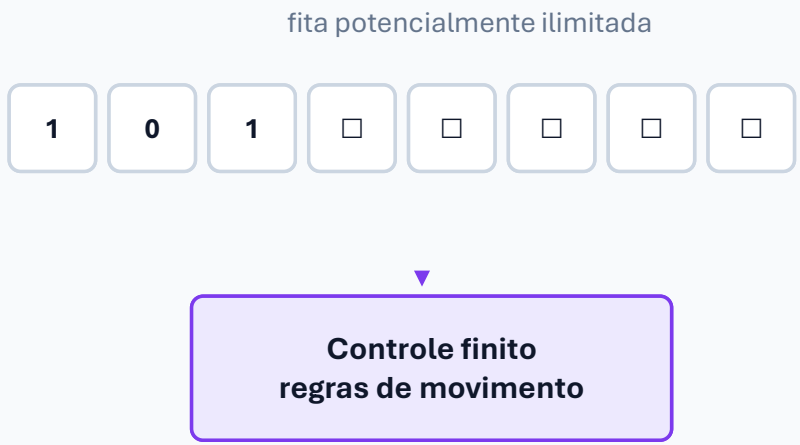
Busca exaustiva
pode crescer muito rápido

Cada escolha de regra cria novos ramos de derivação

Gramáticas irrestritas

Tipo 0

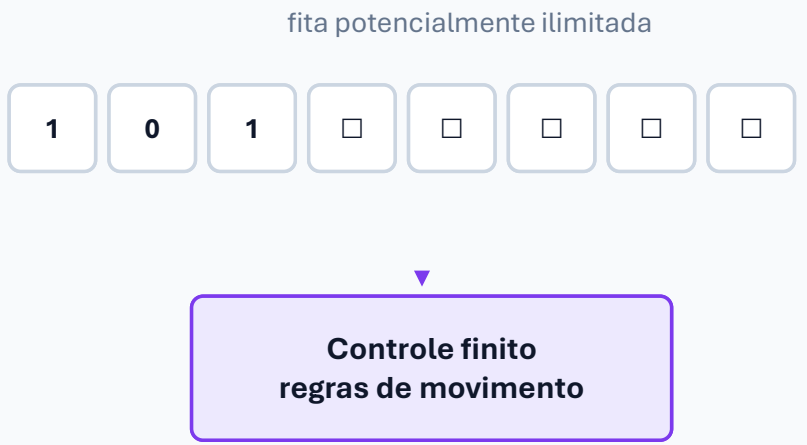
Gramáticas irrestritas permitem regras ainda mais gerais que as sensíveis ao contexto. Nelas, o lado esquerdo da produção pode conter uma sequência não vazia de símbolos. Essa liberdade aumenta o poder de descrição, aproximando o formalismo do comportamento de uma máquina de Turing.



Máquina de Turing: ideia geral

Modelo de computação

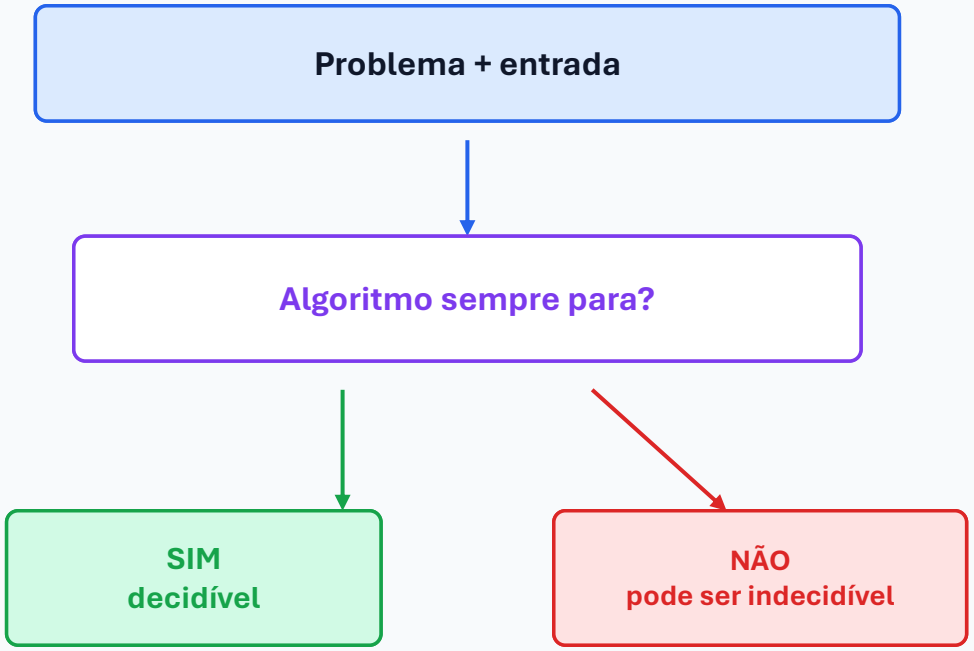
A máquina de Turing é um modelo teórico de computação. Ela possui estados, uma fita, um cabeçote de leitura e escrita, e regras de transição. Apesar de simples, esse modelo representa a ideia de algoritmo de forma muito poderosa e serve para discutir limites computacionais.



Linguagens recursivas

Sempre decide

Uma linguagem recursiva é aquela para a qual existe um algoritmo que sempre termina e responde corretamente se uma cadeia pertence ou não à linguagem. A palavra importante é sempre: para qualquer entrada, o procedimento deve parar com uma resposta sim ou não.



Recursiva × recursivamente enumerável

Diferença essencial

Uma linguagem recursivamente enumerável possui uma máquina que aceita cadeias pertencentes à linguagem, mas pode não parar em entradas que não pertencem. Já uma linguagem recursiva exige parada em todos os casos. Essa diferença é fundamental para entender decidibilidade.



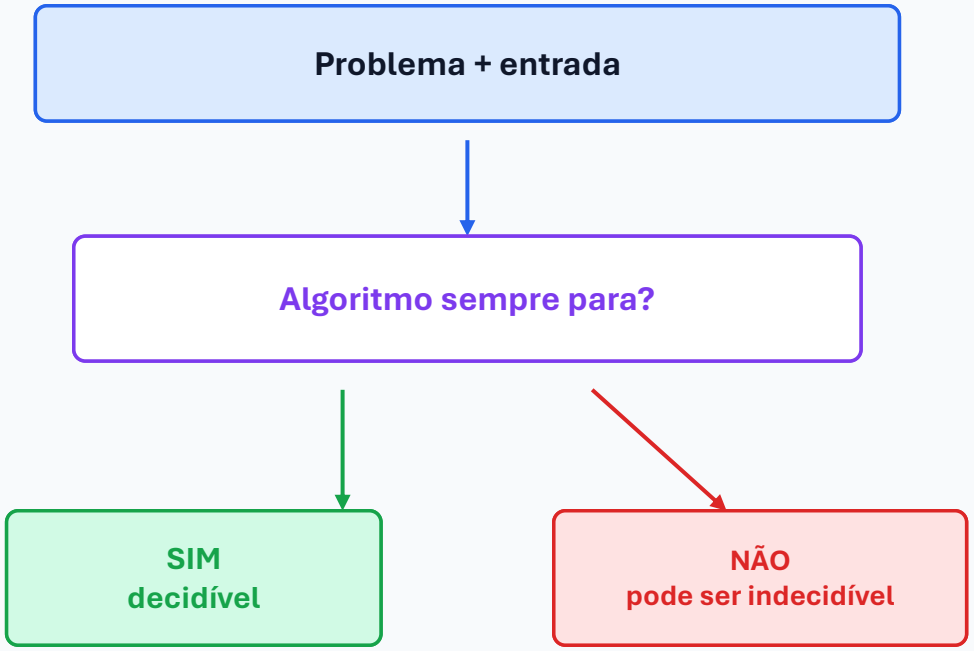
Decidir = sempre terminar com sim ou não

Reconhecer = pode aceitar, mas talvez não pare

Noção de decidibilidade

Perguntas com algoritmo

Um problema é decidível quando existe um algoritmo capaz de responder corretamente a todas as instâncias em tempo finito. A resposta normalmente é sim ou não. Em linguagens formais, isso significa perguntar se uma cadeia pertence a uma linguagem e garantir uma resposta final.



Recurso: YouTube: decidibilidade e problema da parada

Problemas decidíveis em linguagens

Exemplos

Alguns problemas clássicos são decidíveis em classes específicas. Por exemplo, testar se uma cadeia pertence a uma linguagem regular é decidível com autômatos finitos. Testar pertencimento em uma linguagem livre de contexto também é decidível, usando algoritmos de análise sintática.



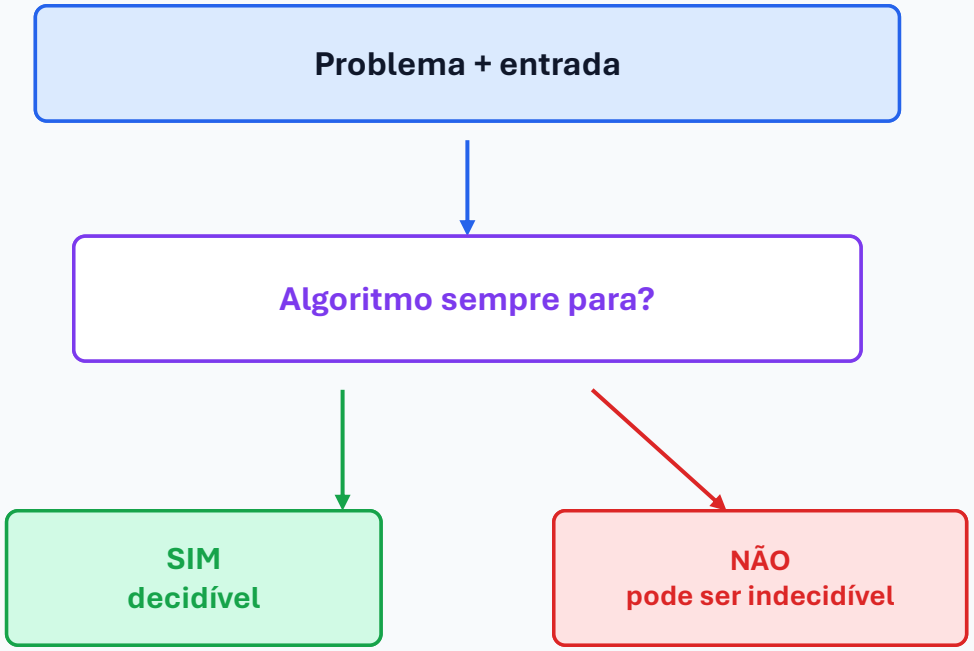
Decidir = sempre terminar com sim ou não

Reconhecer = pode aceitar, mas talvez não pare

Problema da parada

Limite clássico

O problema da parada pergunta se é possível criar um programa que receba outro programa e uma entrada, e diga sempre se esse programa terminará ou ficará executando para sempre. Esse problema é indecidível, mostrando que nem toda pergunta computacional tem algoritmo geral.



Recurso: YouTube: decidibilidade e problema da parada

Decidibilidade e compiladores

Consequências práticas

Compiladores fazem diversas verificações decidíveis, como estrutura sintática, declaração de nomes e compatibilidade básica de tipos. Porém, propriedades gerais sobre o comportamento futuro do programa podem ser indecidíveis. Por isso, ferramentas reais usam regras conservadoras, aproximações e mensagens de aviso.



tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

O que é um compilador?

Definição operacional

Um compilador é um sistema que traduz um programa escrito em uma linguagem fonte para outra representação, geralmente código de máquina, bytecode ou linguagem intermediária. Durante essa tradução, ele analisa o texto, verifica regras, produz diagnósticos e tenta preservar o significado do programa original.



tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

Recurso: LLVM Kaleidoscope – tutorial de front-end

Arquitetura geral do compilador

Pipeline

A arquitetura clássica divide o compilador em front-end, middle-end e back-end. O front-end entende a linguagem fonte; o middle-end trabalha com representação intermediária e otimizações; o back-end gera código para uma plataforma. Essa divisão torna o compilador mais modular.



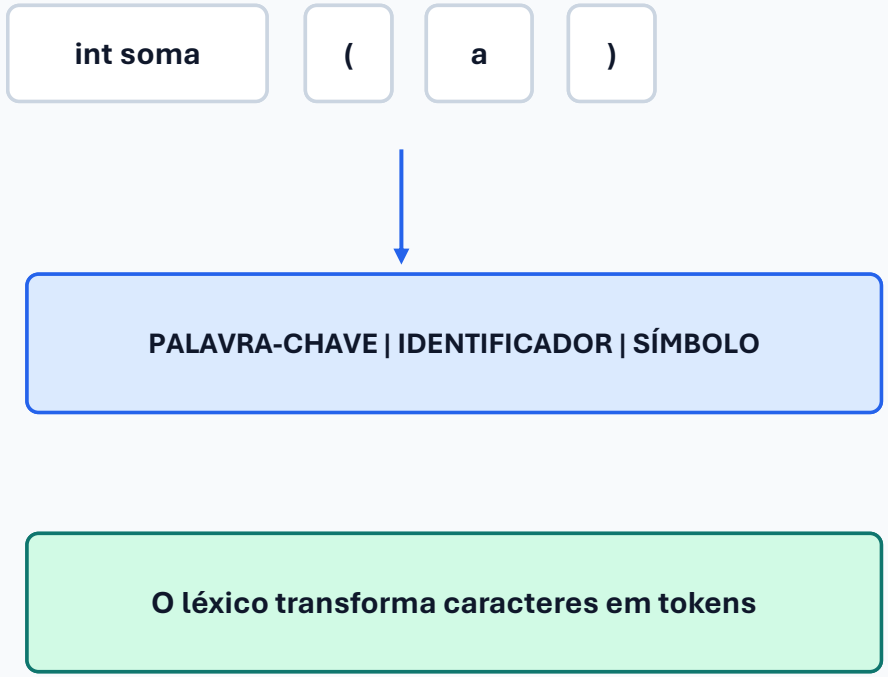
tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

Front-end: entender a linguagem fonte

Entrada do compilador

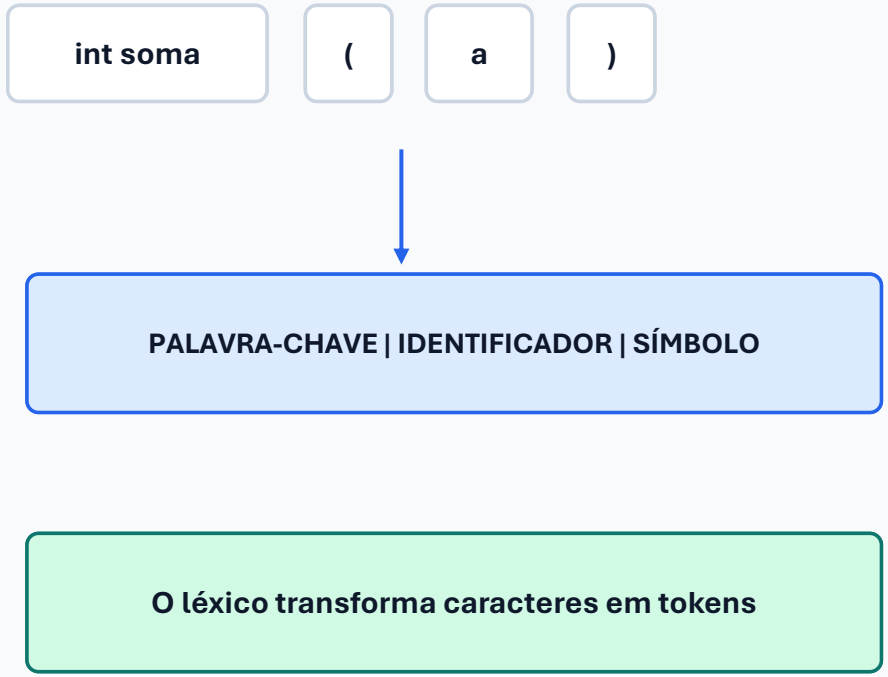
O front-end é responsável por transformar o texto do programa em estruturas compreensíveis pelo compilador. Ele inclui análise léxica, análise sintática e análise semântica. Nessa parte, os conceitos de tokens, gramáticas e árvores sintáticas aparecem de forma muito direta.



Análise léxica

Tokens

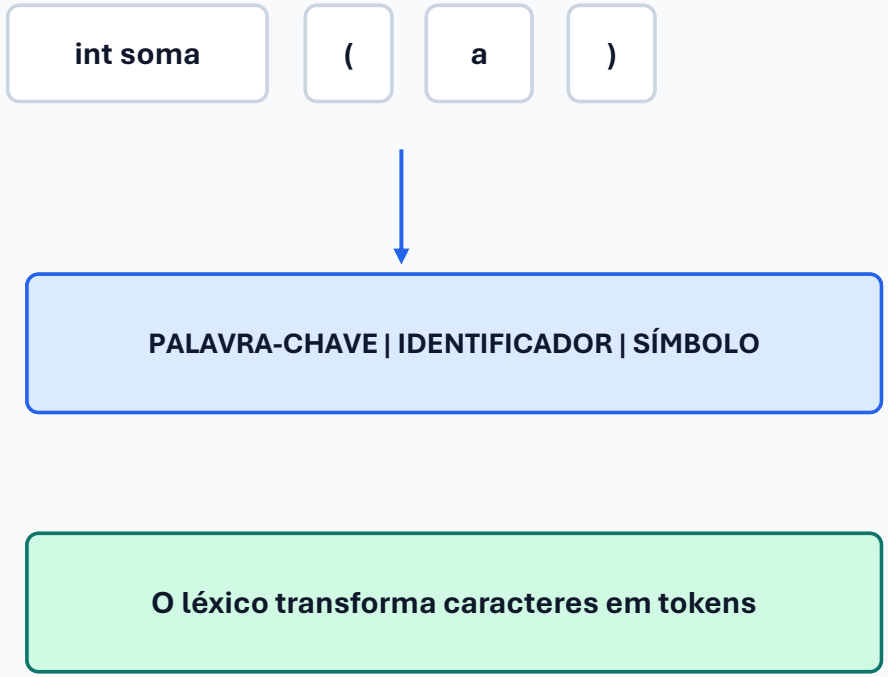
A análise léxica lê caracteres e agrupa sequências em tokens. Um token é uma unidade significativa, como palavra-chave, identificador, número, operador ou delimitador. Por exemplo, em int soma, o lexer reconhece int como palavra-chave e soma como identificador.



Expressões regulares no lexer

Padrões simples

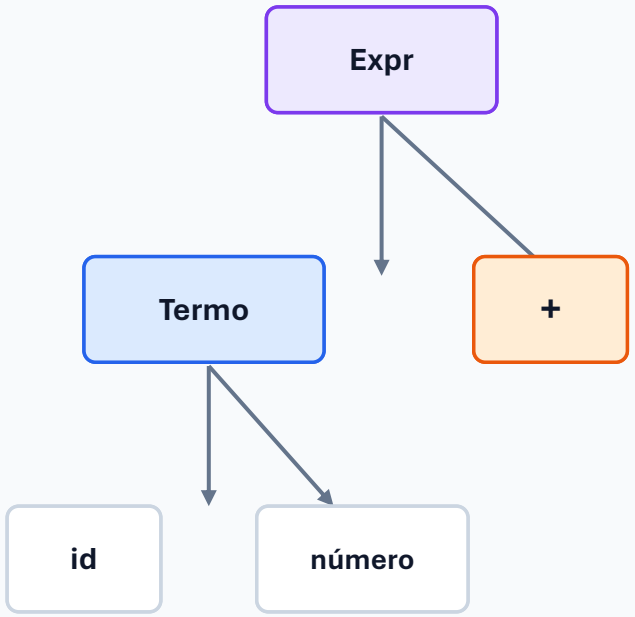
Muitos tokens podem ser descritos por expressões regulares. Identificadores, números inteiros e operadores seguem padrões relativamente locais, sem exigir memória complexa. Por isso, a análise léxica costuma estar fortemente relacionada a autômatos finitos e linguagens regulares.



Análise sintática

Estrutura

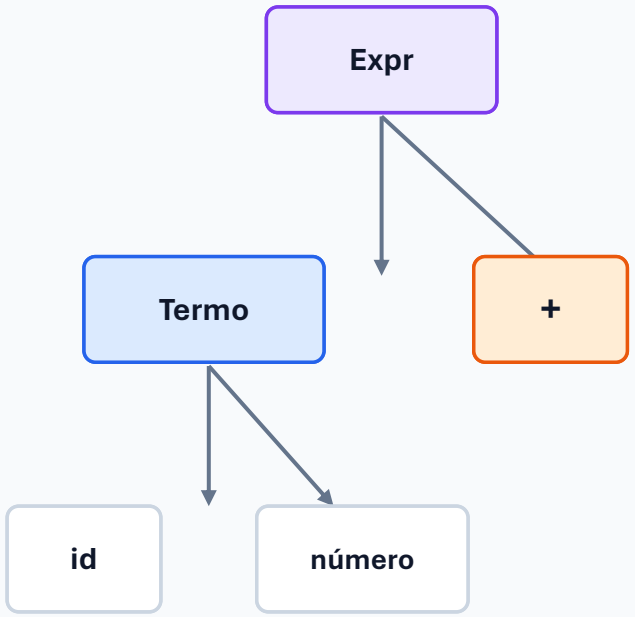
A análise sintática verifica se a sequência de tokens obedece à gramática da linguagem. Ela identifica estruturas como expressões, comandos, blocos e declarações. Quando a entrada é válida, o parser constrói uma árvore sintática ou uma árvore de sintaxe abstrata.



Árvore sintática e AST

Representação

A árvore sintática mostra como uma sentença é derivada pela gramática. A AST, ou árvore de sintaxe abstrata, remove detalhes desnecessários e mantém a estrutura semântica principal. Compiladores usam AST para facilitar verificação, otimização e geração de código.



Análise semântica

Significado

A análise semântica verifica regras que vão além da forma gramatical. Ela confere tipos, escopos, declarações, chamadas de função e usos permitidos de nomes. Um comando pode estar sintaticamente correto e ainda assim ser semanticamente inválido, como somar texto com valor incompatível.



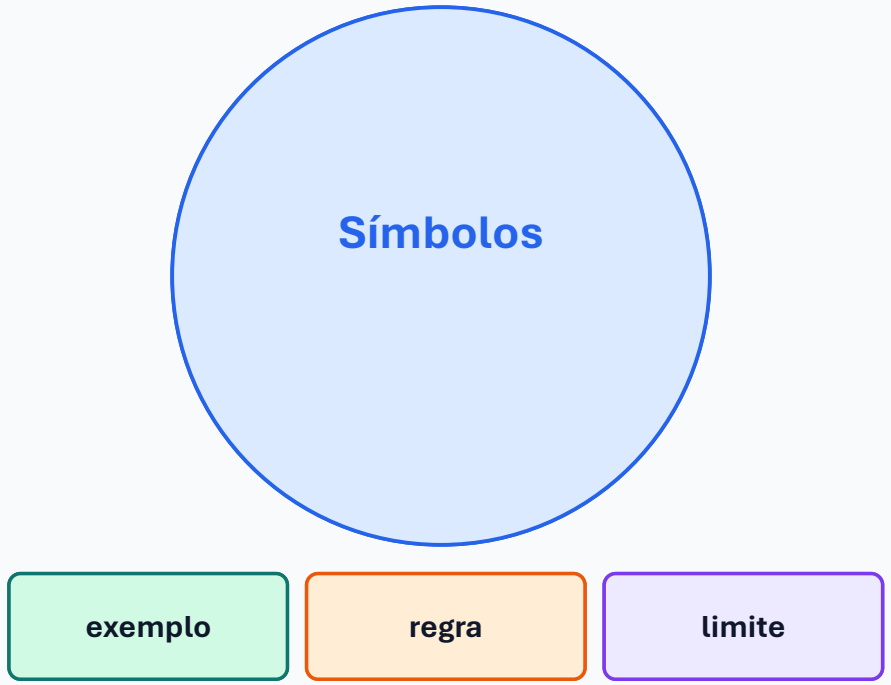
Decidir = sempre terminar com sim ou não

Reconhecer = pode aceitar, mas talvez não pare

Tabela de símbolos

Memória do compilador

A tabela de símbolos registra informações sobre nomes declarados no programa. Ela armazena variáveis, funções, classes, tipos, escopos e atributos necessários para análise semântica. Quando o compilador encontra um identificador, consulta essa tabela para saber se ele existe e como deve ser usado.



Representação intermediária

IR

Após entender o programa, muitos compiladores produzem uma representação intermediária. Ela não é exatamente a linguagem fonte nem o código final da máquina. A IR facilita otimizações e permite separar a análise da linguagem da geração de código para arquiteturas diferentes.



tokens → AST → tipos → otimização → assembly

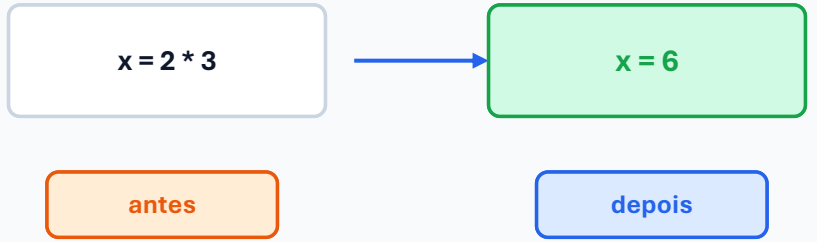
Diagnósticos e tabela de símbolos acompanham as fases

Recurso: LLVM Kaleidoscope – tutorial de front-end

Otimizações

Melhorar sem mudar significado

A otimização busca melhorar o programa sem alterar seu comportamento esperado. Pode remover código morto, simplificar expressões, evitar cálculos repetidos ou reorganizar instruções. O objetivo pode ser reduzir tempo de execução, memória utilizada, consumo de energia ou tamanho do arquivo gerado.



Otimização preserva significado, mas busca menor custo

Back-end e geração de código

Saída

O back-end transforma a representação intermediária em código de destino. Ele considera registradores, instruções disponíveis, convenções da plataforma e organização da memória. Essa etapa aproxima o programa da máquina real, produzindo assembly, objeto, bytecode ou outra forma executável.



tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

Montagem, ligação e execução

Após compilar

Depois da geração de código, ainda podem ocorrer etapas externas ou complementares. O montador transforma assembly em código objeto; o ligador combina módulos e bibliotecas; o carregador prepara o programa para execução. Embora nem sempre sejam fases internas do compilador, fazem parte do caminho até rodar.



tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

Tratamento de erros

Diagnósticos

Um compilador precisa informar erros de forma útil. Erros léxicos envolvem caracteres inesperados; erros sintáticos envolvem estrutura inválida; erros semânticos envolvem significado incorreto. Bons diagnósticos indicam local, causa provável e, quando possível, uma sugestão de correção.



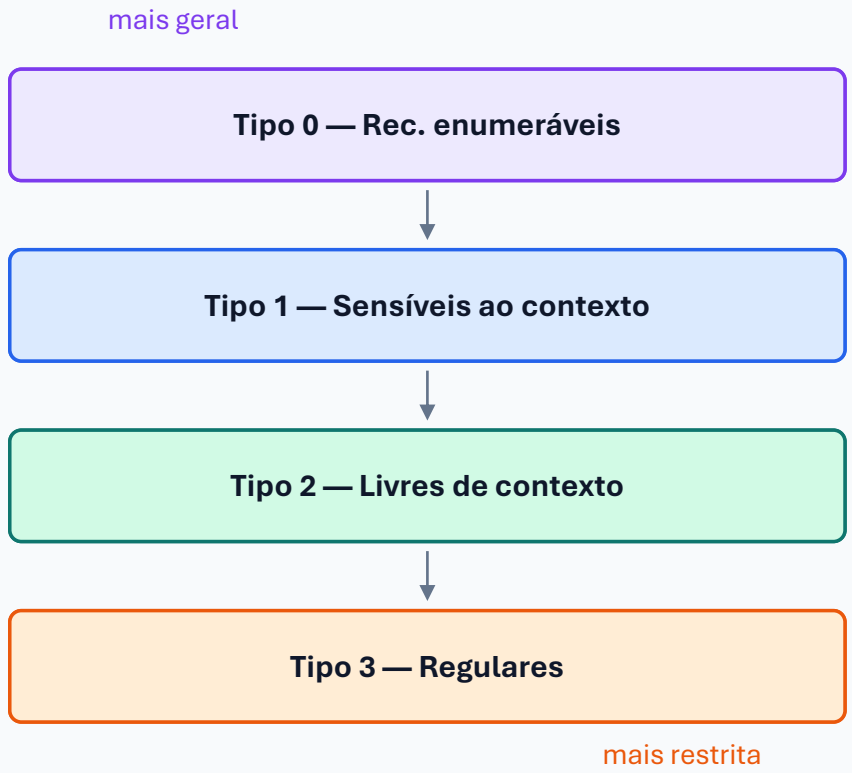
Decidir = sempre terminar com sim ou não

Reconhecer = pode aceitar, mas talvez não pare

Relação entre teoria e prática

Mapa conceitual

As fases do compilador usam ideias de linguagens formais em diferentes níveis. O léxico se relaciona com linguagens regulares; a sintaxe com gramáticas livres de contexto; a semântica envolve regras adicionais; e a decidibilidade mostra limites para verificações automáticas sobre programas.



Passo a passo: observar código compilado

Compiler Explorer

Uma demonstração prática pode ser feita no Compiler Explorer. Abra o site, escolha C ou C++, digite uma função simples, selecione um compilador e observe o assembly gerado. Depois altere opções de otimização, como -O0 e -O2, comparando o código produzido.



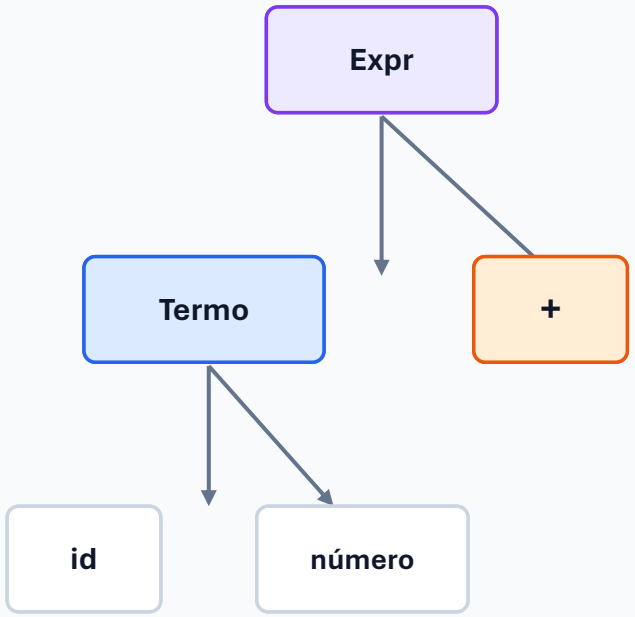
Ferramentas gratuitas ajudam a visualizar conceitos abstratos

Recurso: Compiler Explorer

Passo a passo: tokens e AST com Clang

Demonstração local

Em ambiente com Clang instalado, é possível observar etapas internas. Use comandos para listar tokens ou imprimir a AST de um pequeno programa. Isso ajuda a enxergar que o compilador não “entende” o texto diretamente; ele converte o código em estruturas analisáveis.



Recurso: Clang Internals Manual

Miniestudo: função soma

Do texto ao código

Considere a função int soma(int a, int b) { return a + b; }.

O lexer identifica tokens, o parser reconhece uma definição de função, a semântica verifica tipos inteiros, a IR representa a operação de soma, e o back-end gera instruções para a arquitetura escolhida.



tokens → AST → tipos → otimização → assembly

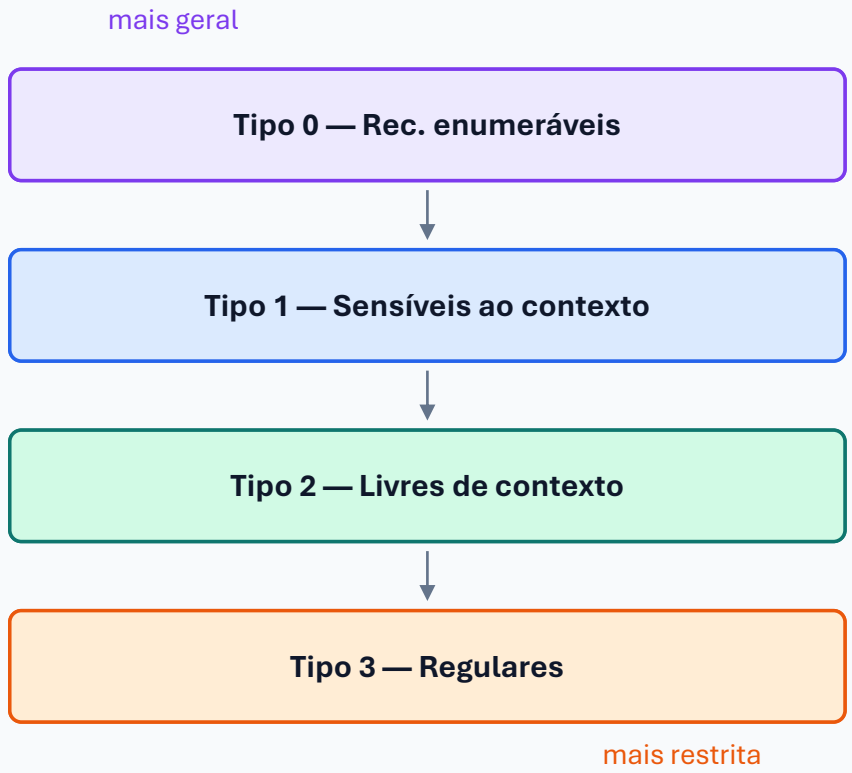
Diagnósticos e tabela de símbolos acompanham as fases

Recurso: YouTube: fases de compiladores

Comparando classes de linguagens

Síntese formal

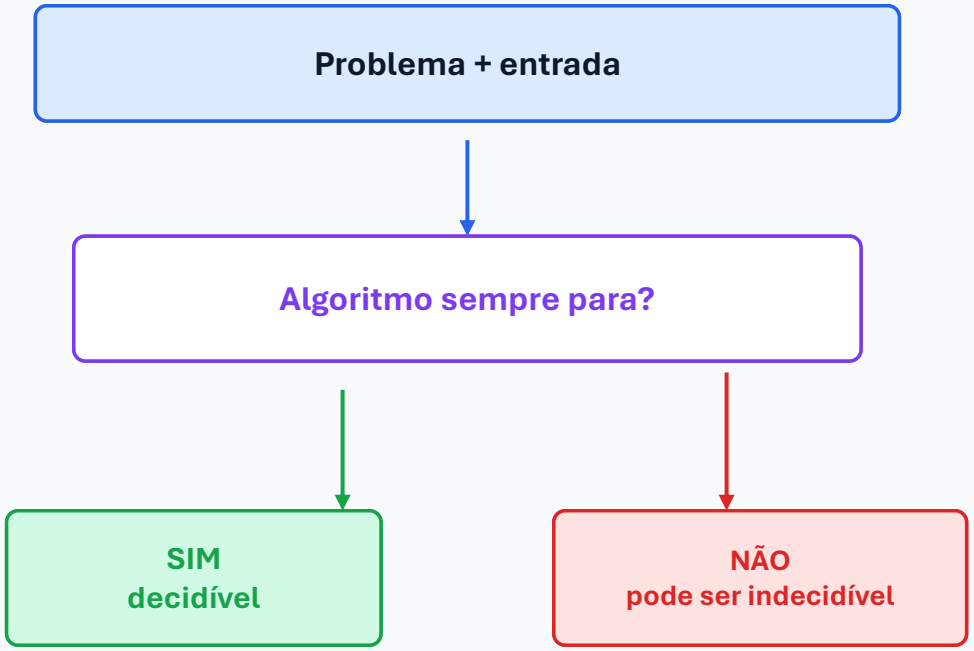
Uma forma simples de organizar o conteúdo é lembrar que cada classe possui regras, reconhecedores e limites próprios. Regulares usam autômatos finitos; livres de contexto usam pilha; sensíveis ao contexto usam memória linear; linguagens recursivas têm decisão garantida por algoritmo que sempre para.



Atividade em sala: classifique

Exercício guiado

Classifique as situações: reconhecer identificadores de uma linguagem, verificar se parênteses estão balanceados, reconhecer $a^n b^n c^n$, decidir se uma cadeia pertence a um autômato finito e determinar se qualquer programa sempre termina. Justifique a classe ou o limite envolvido em cada caso.



Atividade em sala: fases do compilador

Exercício guiado

Organize as etapas para compilar um pequeno programa: ler caracteres, formar tokens, verificar estrutura, construir AST, checar tipos, gerar IR, otimizar, produzir código alvo e ligar bibliotecas. Depois explique o que poderia dar errado em pelo menos três dessas etapas.



tokens → AST → tipos → otimização → assembly

Diagnósticos e tabela de símbolos acompanham as fases

Avaliação formativa

Verificação rápida

Como avaliação rápida, responda: uma regra que diminui o tamanho da cadeia costuma ser adequada para CSG? O que torna uma linguagem recursiva diferente de uma apenas reconhecível? Em qual fase o compilador transforma caracteres em tokens? Em qual fase aparecem AST e regras gramaticais?



Decidir = sempre terminar com sim ou não

Reconhecer = pode aceitar, mas talvez não pare

Avaliação e referências

Fechamento

Avaliação final sugerida: elaborar um mapa relacionando hierarquia de linguagens, decidibilidade e fases de compilação. O estudante deve incluir um exemplo para cada classe discutida e explicar, com suas palavras, onde gramáticas, autômatos e algoritmos aparecem no funcionamento de um compilador.



Ferramentas gratuitas ajudam a visualizar conceitos abstratos